Task 6: MapReduce Application: Implement a MapReduce program to identify "common friends" among all pairs of users

Date of the Session: ----/-----/------ **Time of the Session: 10:20 A.M. to 1:00 P.M.**

Pre Lab:

1. **Elucidate the idea of mutual or common friends within the context of social network analysis?**

2. **Outline the necessary structure of input data for the purpose of identifying common friends through MapReduce?**

3. **Elucidate how the Mapper phase can produce user pairs that facilitate the identification of common friends?**

4. **What is the significance of the Shuffle and Sort phase in addressing the common friend's problem?**

In-Lab:

1. **Execute a Hadoop MapReduce program to identify common friends among all pairs of users.**

STEP 1 Open Cloudera Terminal
STEP 2 : Create a directory in hdfs
hdfs dfs –mkdir /input

```
[cloudera@quickstart ~]$ hdfs dfs -mkdir /input
```

STEP 3 : Create an input file as friends.txt with the following names

```
[cloudera@quickstart ~]$ gedit friends.txt
```

```
[cloudera@quickstart ~]$ cat friends.txt
A:B,C,D
B:A,C,E
C:A,B,D,E
D:A,C,E
E:B,C,D
```

STEP 4 :Upload to HDFS:
Syntax:
hdfs dfs –copyFromLocal friends.txt /user/cloudera/input/friends1.txt

```
[cloudera@quickstart ~]$ hdfs dfs -copyFromLocal friends.txt /user/cloudera/input/friends1.txt
```

```
[cloudera@quickstart ~]$ hdfs dfs -ls /user/cloudera/input
Found 1 items
-rw-r--r--   1 cloudera cloudera         42 2026-02-23 21:34 /user/cloudera/input/friends1.txt
```

STEP 5: Write Java MapReduce program with name SimpleMutualFriends.java
Create SimpleMutualFriends.java with the following content:

```
import java.io.IOException;
import java.util.Arrays;
import java.util.HashSet;
import java.util.Set;
import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.LongWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Job;
import org.apache.hadoop.mapreduce.Mapper;
import org.apache.hadoop.mapreduce.Reducer;
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;

public class SimpleMutualFriends {

    /* ------------------- MAPPER --------------------- */
    public static class MFMapper extends Mapper<LongWritable, Text, Text, Text> {

        Text userPair = new Text();
        Text friends = new Text();
```

```
    public void map(LongWritable key, Text value, Context context)
            throws IOException, InterruptedException {

        String[] parts = value.toString().split(":"); // A:B,C,D
        if (parts.length < 2) return;

        String user = parts[0];
        String[] friendList = parts[1].split(",");

        for (String f : friendList) {

            // always create a sorted pair (A,B) not (B,A)
            String pair = (user.compareTo(f) < 0)
                    ? user + "," + f
                    : f + "," + user;

            userPair.set(pair);
            friends.set(parts[1]); // send full list
            context.write(userPair, friends);
        }
    }
}

/* ------------------- REDUCER ------------------- */
public static class MFReducer extends Reducer<Text, Text, Text, Text> {

    public void reduce(Text key, Iterable<Text> values, Context context)
            throws IOException, InterruptedException {

        String list1 = null;
        String list2 = null;

        // Expect exactly 2 friend lists for each pair
        for (Text t : values) {
            if (list1 == null)
                list1 = t.toString();
            else
                list2 = t.toString();
        }

        if (list1 == null || list2 == null) return;

        // convert to sets
        Set<String> s1 = new HashSet<>(Arrays.asList(list1.split(",")));
        Set<String> s2 = new HashSet<>(Arrays.asList(list2.split(",")));

        // find common friends
        s1.retainAll(s2);

        context.write(key, new Text(s1.toString()));
    }
}

/* ------------------- DRIVER ------------------- */
```

```
    public static void main(String[] args) throws Exception {

        Job job = Job.getInstance(new Configuration(), "Simple Mutual Friends");
        job.setJarByClass(SimpleMutualFriends.class);

        job.setMapperClass(MFMapper.class);
        job.setReducerClass(MFReducer.class);

        job.setOutputKeyClass(Text.class);
        job.setOutputValueClass(Text.class);

        FileInputFormat.addInputPath(job, new Path(args[0]));
        FileOutputFormat.setOutputPath(job, new Path(args[1]));

        System.exit(job.waitForCompletion(true) ? 0 : 1);
    }
}
```

```
[cloudera@quickstart ~]$ cd Friends
[cloudera@quickstart Friends]$ gedit SimpleMutualFriends.java
[cloudera@quickstart Friends]$ cat SimpleMutualFriends.java
import java.io.IOException;
import java.util.Arrays;
import java.util.HashSet;
import java.util.Set;

import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.LongWritable;
```

STEP 6: Compile the Program and Create a jar file

javac –classpath `hadoop classpath` -d . SimpleMutualFriends.java

```
[cloudera@quickstart Friends]$ javac -classpath `hadoop classpath` -d . SimpleMutualFriends.java
```

jar –cvf MutualFriends.jar *.class

```
[cloudera@quickstart Friends]$ jar -cvf MutualFriends.jar *.class
added manifest
adding: SimpleMutualFriends.class(in = 1440) (out= 773)(deflated 46%)
adding: SimpleMutualFriends$MFMapper.class(in = 2120) (out= 929)(deflated 56%)
adding: SimpleMutualFriends$MFReducer.class(in = 2079) (out= 924)(deflated 55%)
[cloudera@quickstart Friends]$
```

STEP 7: Run the jar file using the command

Hadoop jar MutualFriends.jar SimpleMutualFriends /user/cloudera/input/friends1.txt /MutualFriendsOutput

```
[cloudera@quickstart Friends]$ hadoop jar MutualFriends.jar SimpleMutualFriends /user/cloudera/input/friends1.txt /Mu
tualFriendsOutput
26/02/23 22:10:31 INFO client.RMProxy: Connecting to ResourceManager at /0.0.0.0:8032
26/02/23 22:10:32 WARN mapreduce.JobResourceUploader: Hadoop command-line option parsing not performed. Implement the
 Tool interface and execute your application with ToolRunner to remedy this.
26/02/23 22:10:32 INFO input.FileInputFormat: Total input paths to process : 1
26/02/23 22:10:32 WARN hdfs.DFSClient: Caught exception
java.lang.InterruptedException
        at java.lang.Object.wait(Native Method)
        at java.lang.Thread.join(Thread.java:1281)
        at java.lang.Thread.join(Thread.java:1355)
        at org.apache.hadoop.hdfs.DFSOutputStream$DataStreamer.closeResponder(DFSOutputStream.java:967)
        at org.apache.hadoop.hdfs.DFSOutputStream$DataStreamer.endBlock(DFSOutputStream.java:705)
        at org.apache.hadoop.hdfs.DFSOutputStream$DataStreamer.run(DFSOutputStream.java:894)
26/02/23 22:10:32 WARN hdfs.DFSClient: Caught exception
java.lang.InterruptedException
        at java.lang.Object.wait(Native Method)
        at java.lang.Thread.join(Thread.java:1281)
        at java.lang.Thread.join(Thread.java:1355)
        at org.apache.hadoop.hdfs.DFSOutputStream$DataStreamer.closeResponder(DFSOutputStream.java:967)
        at org.apache.hadoop.hdfs.DFSOutputStream$DataStreamer.endBlock(DFSOutputStream.java:705)
        at org.apache.hadoop.hdfs.DFSOutputStream$DataStreamer.run(DFSOutputStream.java:894)
26/02/23 22:10:32 INFO mapreduce.JobSubmitter: number of splits:1
26/02/23 22:10:32 INFO mapreduce.JobSubmitter: Submitting tokens for job: job_1771910702594_0001
26/02/23 22:10:33 INFO impl.YarnClientImpl: Submitted application application_1771910702594_0001
26/02/23 22:10:33 INFO mapreduce.Job: The url to track the job: http://quickstart.cloudera:8088/proxy/application_177
1910702594_0001/
26/02/23 22:10:33 INFO mapreduce.Job: Running job: job_1771910702594_0001
26/02/23 22:10:40 INFO mapreduce.Job: Job job_1771910702594_0001 running in uber mode : false
26/02/23 22:10:40 INFO mapreduce.Job:  map 0% reduce 0%
26/02/23 22:10:45 INFO mapreduce.Job:  map 100% reduce 0%
26/02/23 22:10:51 INFO mapreduce.Job:  map 100% reduce 100%
26/02/23 22:10:52 INFO mapreduce.Job: Job job_1771910702594_0001 completed successfully
26/02/23 22:10:52 INFO mapreduce.Job: Counters: 49
        File System Counters
                FILE: Number of bytes read=206
                FILE: Number of bytes written=286825
                FILE: Number of read operations=0
                FILE: Number of large read operations=0
                FILE: Number of write operations=0
                HDFS: Number of bytes read=171
                HDFS: Number of bytes written=76
                HDFS: Number of read operations=6
                HDFS: Number of large read operations=0
                HDFS: Number of write operations=2
        Job Counters
                Launched map tasks=1
                Launched reduce tasks=1
```

```
                FILE: Number of write operations=0
                HDFS: Number of bytes read=171
                HDFS: Number of bytes written=76
                HDFS: Number of read operations=6
                HDFS: Number of large read operations=0
                HDFS: Number of write operations=2
        Job Counters
                Launched map tasks=1
                Launched reduce tasks=1
                Data-local map tasks=1
                Total time spent by all maps in occupied slots (ms)=2917
                Total time spent by all reduces in occupied slots (ms)=3725
                Total time spent by all map tasks (ms)=2917
                Total time spent by all reduce tasks (ms)=3725
                Total vcore-milliseconds taken by all map tasks=2917
                Total vcore-milliseconds taken by all reduce tasks=3725
                Total megabyte-milliseconds taken by all map tasks=2987008
                Total megabyte-milliseconds taken by all reduce tasks=3814400
        Map-Reduce Framework
                Map input records=5
                Map output records=16
                Map output bytes=168
                Map output materialized bytes=206
                Input split bytes=129
                Combine input records=0
                Combine output records=0
                Reduce input groups=8
                Reduce shuffle bytes=206
                Reduce input records=16
                Reduce output records=8
                Spilled Records=32
                Shuffled Maps =1
                Failed Shuffles=0
                Merged Map outputs=1
                GC time elapsed (ms)=78
                CPU time spent (ms)=690
                Physical memory (bytes) snapshot=362790912
                Virtual memory (bytes) snapshot=3015073792
                Total committed heap usage (bytes)=226365440
        Shuffle Errors
                BAD_ID=0
                CONNECTION=0
                IO_ERROR=0
                WRONG_LENGTH=0
                WRONG_MAP=0
                WRONG_REDUCE=0
        File Input Format Counters
                Bytes Read=42
        File Output Format Counters
                Bytes Written=76
[cloudera@quickstart Friends]$
```

STEP 8: Check the output in the output folder

```
[cloudera@quickstart Friends]$ hdfs dfs -ls /MutualFriendsOutput
Found 2 items
-rw-r--r--   1 cloudera supergroup          0 2026-02-23 22:10 /MutualFriendsOutput/_SUCCESS
-rw-r--r--   1 cloudera supergroup         76 2026-02-23 22:10 /MutualFriendsOutput/part-r-00000
[cloudera@quickstart Friends]$
```

hdfs dfs –cat /MutualFreindsOutput/part-r-00000

```
[cloudera@quickstart Friends]$ hdfs dfs -cat /MutualFriendsOutput/part-r-00000
A,B     [C]
A,C     [D, B]
A,D     [C]
B,C     [E, A]
B,E     [C]
C,D     [E, A]
C,E     [D, B]
D,E     [C]
```

POST LAB:

1). Describe the process by which the reducer logic determines shared friends between two users.

2). What occurs if a user does not share any friends with another user?

3) In what way does the quantity of mappers and reducers influence performance when handling large datasets?

4). Propose one optimization method to enhance the efficiency of the common friends MapReduce program.

(For Evaluator's use only)

Comment of the Evaluator (if any)	Evaluator's Observation Marks Secured:_____out of _____ Signature of the Evaluator